Lets build our mental model before learning ROS:

Problem	ROS solution
Need to pass messages between processes	**Topics** (pub/sub, real-time safe, typed)
Need services (RPC-style)	**Services** (request-response)
Need long-running state sync (like heartbeat, status, config)	**Parameters**
Need to visualize sensors, joints, transforms	**RViz**
Need to describe your robot geometry	**URDF (Unified Robot Description Format)**
Need trajectory planning or kinematics solvers	**MoveIt**, **ros2_control**, **KDL**, etc.
Need logging, time sync, bag replay	Built-in tools
Need simulation	Gazebo, IsaacSim, etc.

ROS2 Concept	Analogy in ML / Systems Engineering
Node	Microservice or script process
Topic	Pub/sub message channel (Kafka/redis topic)
Message	Typed protobuf-like(or json) data structure
Publisher	Producer
Subscriber	Consumer
Service	RPC endpoint
Action	Async task with progress feedback (good for “move arm to X”)
Launch file	Orchestration YAML that starts multiple nodes
TF tree	Coordinate frame graph (how sensors relate spatially)

Regarding Kinematic Equations: **math engine under ROS**, used when you want to compute:

- Forward kinematics: “If each joint is at θ_1, θ_2, … → where’s the gripper?”
- Inverse kinematics: “If I want the gripper at (x, y, z, roll, pitch, yaw) → what are the joint angles?”
- Motion planning: smooth, collision-free joint trajectories.

These are handled via **URDF + MoveIt + ros2_control**, not by you manually coding equations.

you’ll feed your **robot model (URDF)**, and ROS can give you these:

- transform frames,

- coordinate conversions,
- motion planning pipeline.

SETUP(on windows machine)

1. Install the ubuntu on ws

- `wsl --install -d Ubuntu-22.04`
- `wsl --set-default Ubuntu-22.04`
- `wsl -d Ubuntu-22.04`
- `sudo apt update && sudo apt upgrade -y`
- `sudo apt install -y curl gnupg lsb-release`

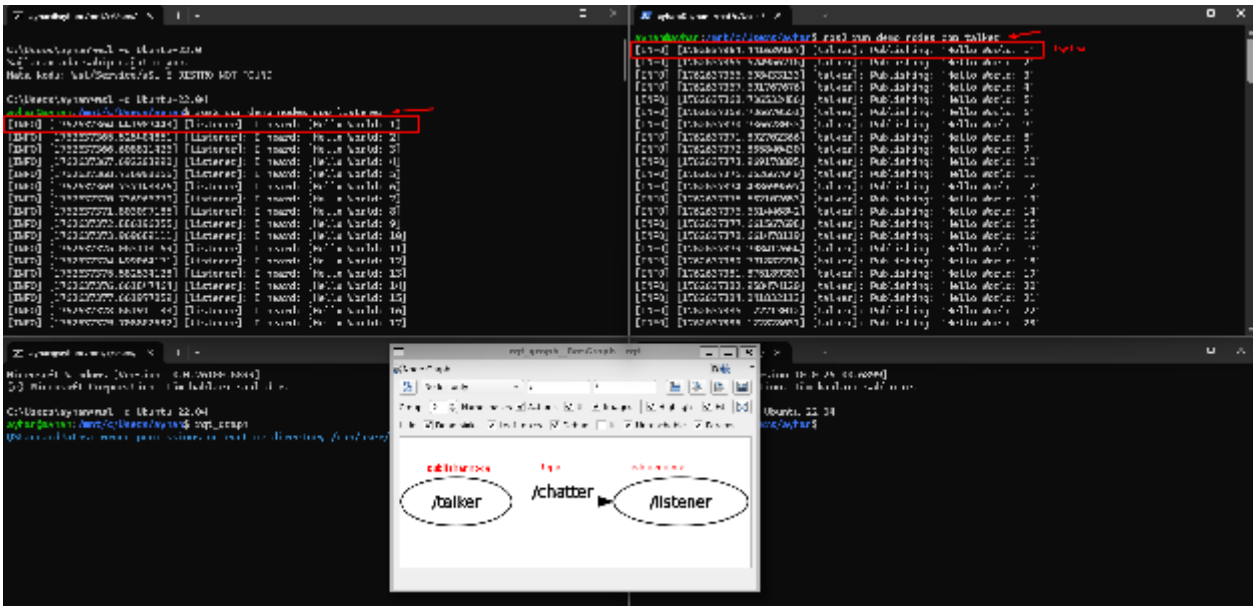
2. Now lets turn this clean Ubuntu into a ROS2 Humble development machine

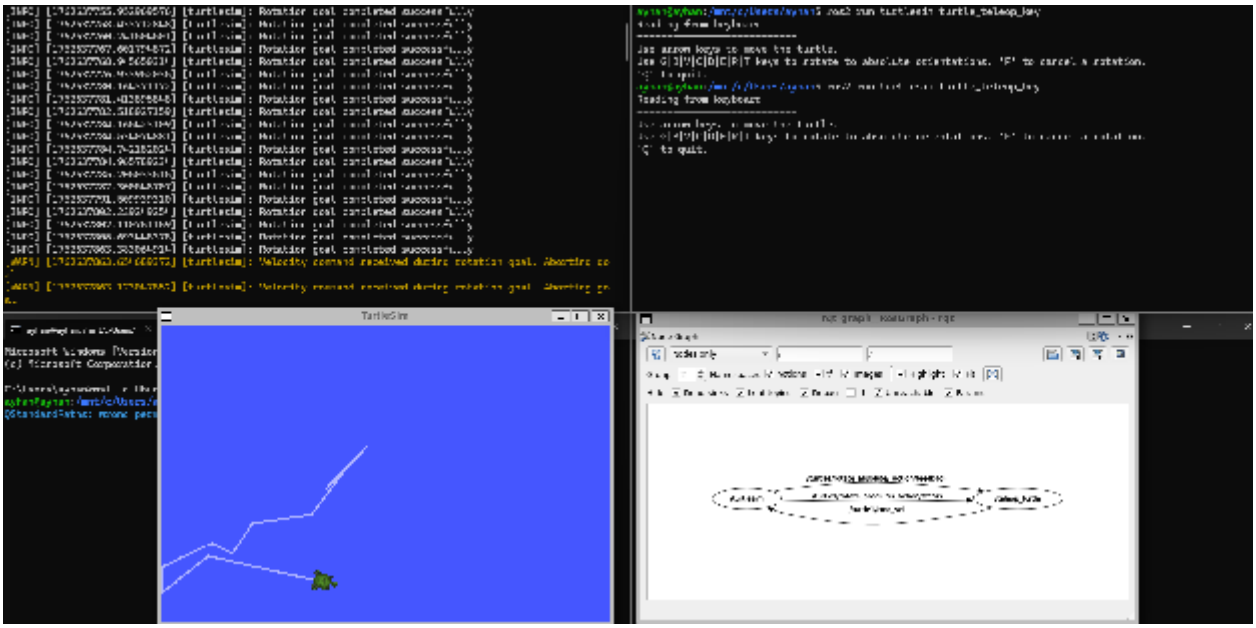
- `sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg `
- `echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null `
- `sudo apt update`
- `sudo apt install -y ros-humble-desktop`
- `echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc`
- `source ~/.bashrc`
- Check by the commands : `printenv | grep ROS`& ls /opt/ros/

You should see

```
ayhan@ayhan:/mnt/c/Users/ayhan$ ls /opt/ros/
humble
ayhan@ayhan:/mnt/c/Users/ayhan$ printenv | grep ROS
ROS_VERSION=2
ROS_PYTHON_VERSION=3
ROS_LOCALHOST_ONLY=0
ROS_DISTRO=humble
ayhan@ayhan:/mnt/c/Users/ayhan$ ros2 topic list
/parameter_events
/rosout
ayhan@ayhan:/mnt/c/Users/ayhan$
```

- Finally, run a demo by: `ros2 run demo_nodes_cpp talker` & `ros2 run demo_nodes_cpp talker` and `rqt_graph`





Now after the setup and testing demo apps, lets start our ros workspace to keep things organized. Its like custom packages in python.

We'll need a build tool namely colcon

- `sudo apt install python3-colcon-common-extensions`
- Enable colcon-autocomplete by: `echo 'eval "$(register-python-argcomplete3 colcon)"' >> ~/.bashrc` then -> `source ~/.bashrc` -> then test with `colcon bu<TAB>` to see it autocompletes to build

Then create and cd into the workspace : `mkdir harvesting_ws` -> `cd harvesting_ws` -> `mkdir src`
Run the build to introduce it as ros2 package: `colcon build`

Now we'll have build, install and log files and it will look into the src/ for our custom logic.

```
ayhan@ayhan:/mnt/c/Users/ayhan/harvesting_ws$ ls
build  install  log  src
ayhan@ayhan:/mnt/c/Users/ayhan/harvesting_ws$
```

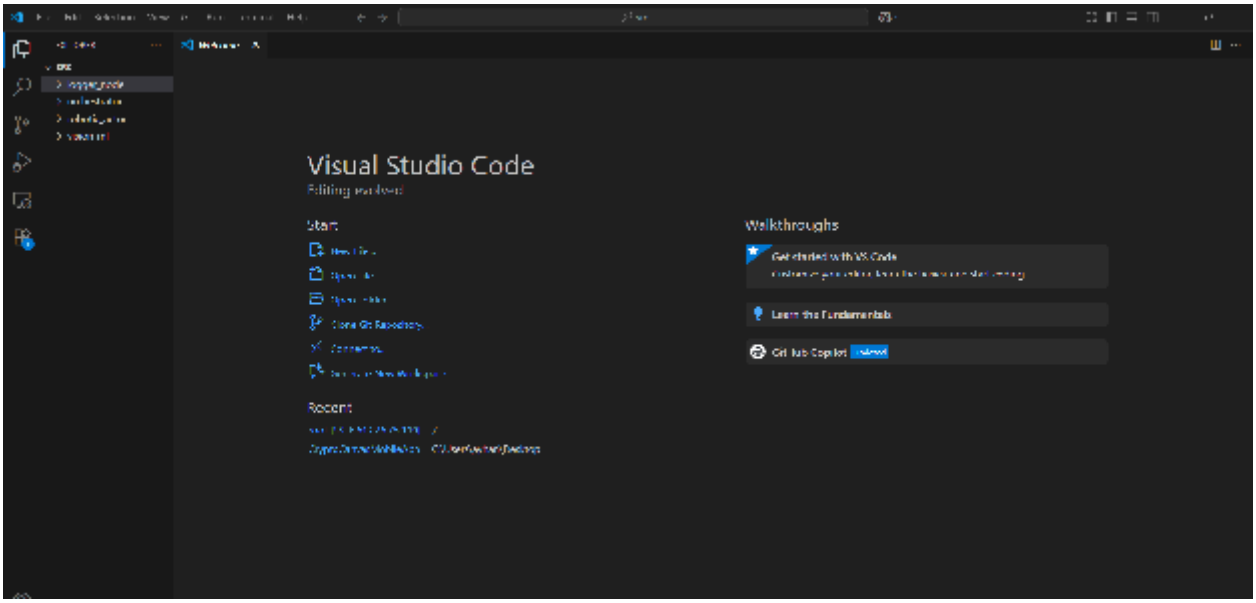
Now lets make sure our **workspace environment (`install/setup.bash`)** loads automatically so you don't have to `source` it manually every time.
`source /mnt/c/Users/ayhan/harvesting_ws/install/setup.bash`
Verify by : `echo $ROS_DISTRO`

Now finally, we'll have to create our python package in the src/ folder. Run all four, which are main components in our design:

`ros2 pkg create --build-type ament_python orchestrator --dependencies rclpy std_msgs
ros2 pkg create --build-type ament_python vision_ml --dependencies rclpy sensor_msgs std_msgs
ros2 pkg create --build-type ament_python robotic_actor --dependencies rclpy geometry_msgs std_msgs
ros2 pkg create --build-type ament_python logger_node --dependencies rclpy std_msgs
`

This will give us a folder structure like:
src/
├─ orchestrator/
├─ vision_ml/
├─ robotic_actor/
└─ logger_node/

Install vscode (or any ide of your choice): `sudo snap install code --classic`
Then run `code .` to open it up.



I've created demo for our problem:

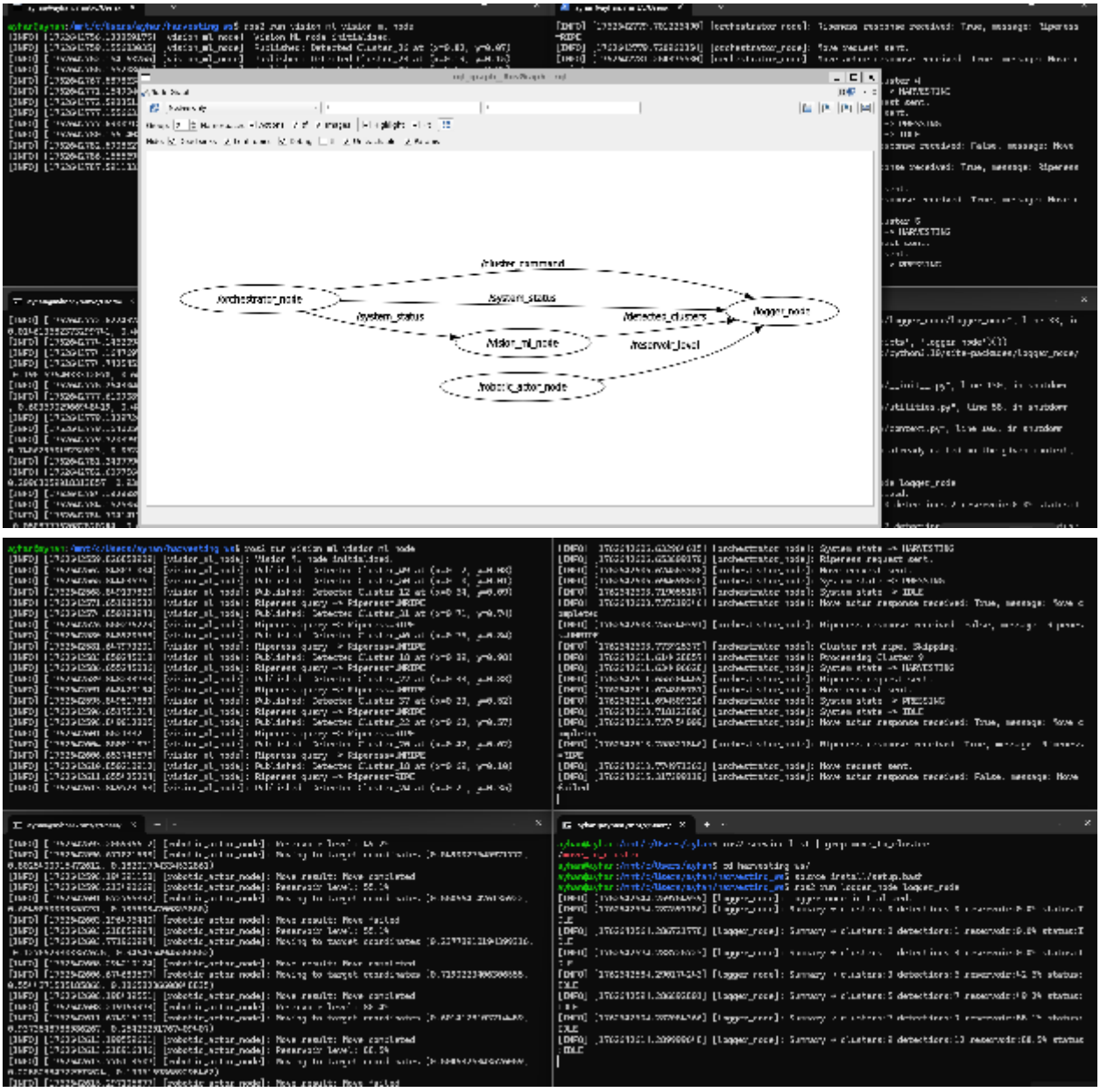
After having all 4 packages build with colcon, we can run the simulation following these steps:

1. Open up 4 terminals, run wsl -d Ubuntu-22.04 to enter the linux distro
2. Cd into harvesting_ws (workspace)
3. Run source install/setup.bash in each, this will set up the shell environment so ROS 2 can find and use our packages.
4. Then launch the full system(one for each terminal):

    1. `ros2 run vision_ml vision_ml_node`

2. `ros2 run robotic_actor robotic_actor_node`
3. `ros2 run orchestrator orchestrator_node`
4. `ros2 run logger_node logger_node`

5. We can open up another terminal to see the graph(how all our nodes, topics, services, and actions are connected.): rqt_graph



Reservoir demo: **ros2 launch robotic_actor display_four_bar.launch.py**

MODEL -> URDF -> RVIZ

ROS2 URDF Tutorial - Describe Any Robot (Links and Joints)
Create a URDF with ROS2 [1H Crash Course]
How do we describe a robot? With URDF! | Getting Ready to build Robots with ROS #7

creating urdf file seems messy when done manually, so we should search for SolidWorks -> urdf plugins, i guess there are some e.g. http://wiki.ros.org/sw_urdf_exporter an official add-in from the ROS community.

Basic Steps:

1. Design your robot assembly in SolidWorks with proper mates
2. Tools → Export as URDF
3. Define which parts are links and which mates are joints
4. Set joint types (revolute, prismatic, fixed, etc.)
5. Export → generates URDF + STL/DAE meshes

Solidworks to URDF file for ros2

Convert SolidWorks to URDF for ROS2 - Full Tutorial

We can go with Onshape too, a browser-based, free educational use tool.

export LIBGL_ALWAYS_SOFTWARE=1